

Documentación Futboleros

Índice Importante:

Links:

- [Links útiles](#)

Código

- [Detección de mandos](#)
- [Desconexión de mandos](#)
- [Testeo de mandos por monitor serial](#)
- [Detección de soporte de mandos](#)
- [Emparejamiento de un solo mando](#)

Bluepad32 ESP32

Links Útiles:

- Mandos soportados: https://bluepad32.readthedocs.io/en/latest/supported_gamepads/
- Emparejar un solo mando: <https://bluepad32.readthedocs.io/en/latest/FAQ/#how-to-pair-just-one-controller-to-one-particular-board>

✓ Ventajas sobre el PS4 Controller.

	Bluepad32	PS4 Controller
Compatibilidad	Genérico	Sólo para PS4
Complejidad de emparejamiento	Fácil y de muchos recursos	Complejo
Mantenimiento	Actualizaciones constantes	Librería obsoleta

Debemos agregarle verificación / autenticación. Se explica en el último link, pero se debe adicionar la mejora de la selección del mismo posiblemente con web server.

Código Machetes sobre el Example:

```
#include <Bluepad32.h> // Librería
```

```
// Creación de la Estructura =====  
ControllerPtr myControllers [BP32_MAX_GAMEPADS];
```

Detección de mandos:

```
// Función para Detectar Conexión =====  
void onConnectedController(ControllerPtr ctl) { // Se crea un objeto ctl
```

```

// Se inicializa al Slot a emparejar como vacío.
    bool foundEmptySlot = false;

// Se recorre la cantidad máxima de mandos a conectar (hasta 4 a la vez)
    for (int i = 0; i < BP32_MAX_GAMEPADS; i++) {
        // Si hay slots libres, se conecta el mando:
        if (myControllers[i] == nullptr) {
            Serial.printf("CALLBACK: Controller is connected, index=%d\n", i);
            // Creo objeto propiedades y obtengo las mismas
            ControllerProperties properties = ctl->getProperties();
            // Imprimo características de nuestro mando 'ctl'
            Serial.printf("Controller model: %s, VID=0x%04x, PID=0x%04x\n", ctl-
>getModelName().c_str(), properties.vendor_id, properties.product_id);
            // Le adjudico al slot libre, mi mando
            myControllers[i] = ctl;
            // Le paso 'true' a mi valor centinela que determina si hay slots libres; de esta
manera.
                foundEmptySlot = true;
                break;
        }
    }
    if (!foundEmptySlot) {
        Serial.println("CALLBACK: Controller connected, but could not found empty slot");
    }
}

```

Desconexión de mandos:

```

// Función de Desconexión de Mando =====
// Es la misma función de conexión, pero borro con 'nullptr' el mando de la lista de mandos
conectados.
void onDisconnectedController(ControllerPtr ctl) {

    bool foundController = false;

    for (int i = 0; i < BP32_MAX_GAMEPADS; i++) {
        if (myControllers[i] == ctl) {
            Serial.printf("CALLBACK: Controller disconnected from index=%d\n", i);
            // Borro las características del mando de la lista de mandos conectados
            myControllers[i] = nullptr;
            // 'true' a eliminar el mando
            foundController = true;
            break;
        }
    }

    if (!foundController) {
        Serial.println("CALLBACK: Controller disconnected, but not found in myControllers");
    }
}

```

Testeo de Mando por Monitor Serial:

```

void dumpGamepad(ControllerPtr ctl) {
    Serial.printf( "a:%d, b:%d, x:%d, y:%d, axis L: %4d, %4d, axis R: %4d, %4d, stickL:%d,
stickR: %d, l2:%d, r2:%d, l1: %d, r1: %d \n",
        ctl->a(), // True si A es presionado
        ctl->b(), // True si B es presionado
        ctl->x(), //True si X es presionado
        ctl->y(), // True si Y es presionado
        ctl->axisX(), // (-511 - 512) Eje X Izquierdo
        ctl->axisY(), // (-511 - 512) Eje Y Izquierdo
        ctl->axisRX(), // (-511 - 512) Eje X Derecho
        ctl->axisRY(), // (-511 - 512) Eje Y Derecho
        ctl->thumbL(), // Botón de Stick Izquierdo
        ctl->thumbR(), // Botón de Stick Derecho
        // Botones de R1, R2, L1, L2
        ctl->l1(),
        ctl->r1(),
        ctl->brake(), // (0 - 1023)
        ctl->throttle() // (0 - 1023)
    );
}

```

Detección de Soporte de Mando:

```

void processControllers() {
    for (auto myController : myControllers) {
        if (myController && myController->isConnected() && myController->hasData()) {
            if (myController->isGamepad()) {
                processGamepad(myController);
            } else {
                Serial.println("Unsupported controller");
            }
        }
    }
}

```

Algunos Comandos en Setup() :

```

// Setear los callbacks del Bluepad32
BP32.setup(&onConnectedController, &onDisconnectedController);

// El siguiente comando permite borrar la lista de conexiones para reset.
BP32.forgetBluetoothKeys();

```

Función Loop ():

```

void loop() {
    bool dataUpdated = BP32.update();
    if (dataUpdated)
        processControllers();
    delay(150);
}

```

Emparejamiento de un solo Mando:

<https://bluepad32.readthedocs.io/en/latest/FAQ/#how-to-pair-just-one-controller-to-one-particular-board>

```
#include <uni.h> // Librería
```

```
// Se agrega una variable con la dirección Bluetooth del mando  
static const char * controller_addr_string = "00:11:22:33:44:55";
```

Algunos comandos en el Setup ()

```
bd_addr_t controller_addr;  
  
// Analizar la dirección Bluetooth legible por humanos  
sscanf_bd_addr(controller_addr_string, controller_addr);  
  
// Agregar la dirección del control a la lista de permitidos  
uni_bt_allowlist_add_addr(controller_addr);  
  
// Enable de  
uni_bt_allowlist_set_enabled(true);
```

NVS:

Notar que la dirección será agregada a la Non-volatile-storage (NVS).

De esta manera, si el dispositivo se reinicia, la dirección permanecerá almacenada.
